



25  
SOICT

HA NOI UNIVERSITY OF SCIENCE AND TECHNOLOGY  
SCHOOL OF INFORMATION AND COMMUNICATION TECHNOLOGY

# Computer Vision

Chapter 7 (part 3):  
Deep Learning for CV

## Deep learning for CV



25  
SOICT

SCHOOL OF INFORMATION AND COMMUNICATION TECHNOLOGY

## Contents

1. Object detection: sliding-windows
2. Two-stage object detection
3. One-stage object detection: Anchor-based
4. One-stage object detection: Anchor-free
5. Semantic segmentation
6. Instance segmentation



SCHOOL OF INFORMATION AND COMMUNICATION TECHNOLOGY

3

## Computer Vision Tasks

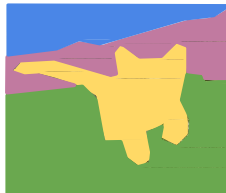
**Classification**



**CAT**

No spatial extent

**Semantic Segmentation**



**GRASS, CAT,  
TREE, SKY**

No objects, just pixels

**Object Detection**



**DOG, DOG, CAT**

Multiple Object

**Instance Segmentation**



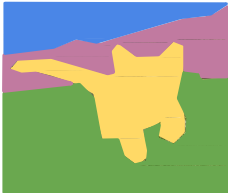
**DOG, DOG, CAT**



SCHOOL OF INFORMATION AND COMMUNICATION TECHNOLOGY

4

## Semantic Segmentation

| Classification                                                                    | Semantic Segmentation                                                             | Object Detection                                                                   | Instance Segmentation                                                               |
|-----------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
|  |  |  |  |
| CAT                                                                               | GRASS, CAT, TREE, SKY                                                             | DOG, DOG, CAT                                                                      | DOG, DOG, CAT                                                                       |
| No spatial extent                                                                 | No objects, just pixels                                                           | Multiple Object                                                                    |                                                                                     |

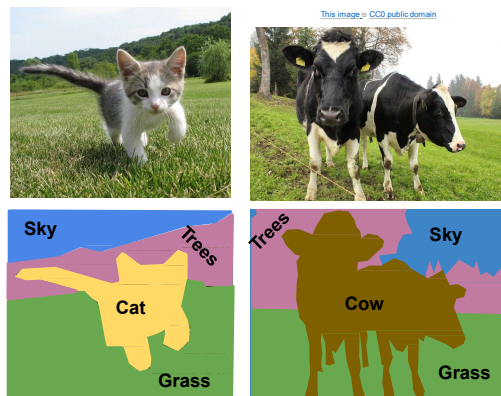

 SCHOOL OF INFORMATION AND COMMUNICATION TECHNOLOGY

5

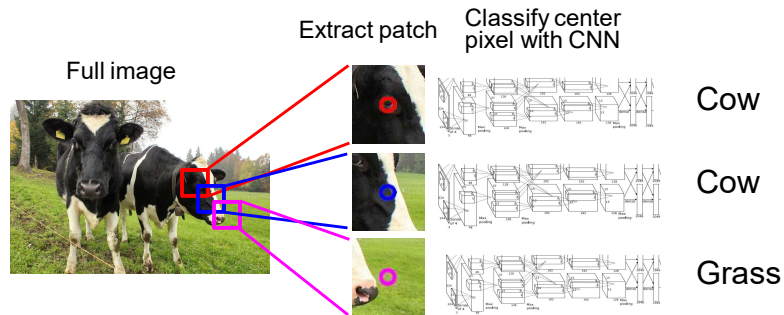
## Semantic Segmentation

Label each pixel in the image with a category label

Don't differentiate instances, only care about pixels



## Semantic Segmentation Idea: Sliding Window

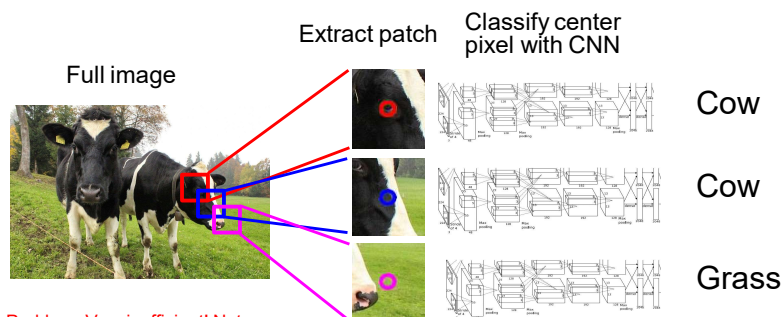


Farabet et al, "Learning Hierarchical Features for Scene Labeling," TPAMI 2013  
 Pinheiro and Collobert, "Recurrent Convolutional Neural Networks for Scene Labeling", ICML 2014

SCHOOL OF INFORMATION AND COMMUNICATION TECHNOLOGY

7

## Semantic Segmentation Idea: Sliding Window



Problem: Very inefficient! Not reusing shared features between overlapping patches



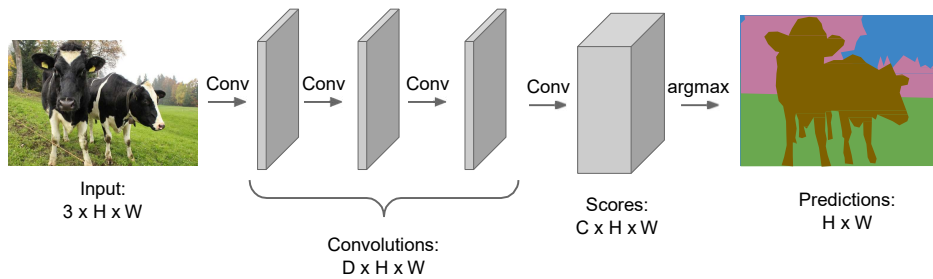
Farabet et al, "Learning Hierarchical Features for Scene Labeling," TPAMI 2013  
 Pinheiro and Collobert, "Recurrent Convolutional Neural Networks for Scene Labeling", ICML 2014

SCHOOL OF INFORMATION AND COMMUNICATION TECHNOLOGY

8

## Semantic Segmentation Idea: Fully Convolutional

Design a network as a bunch of convolutional layers to make predictions for pixels all at once!

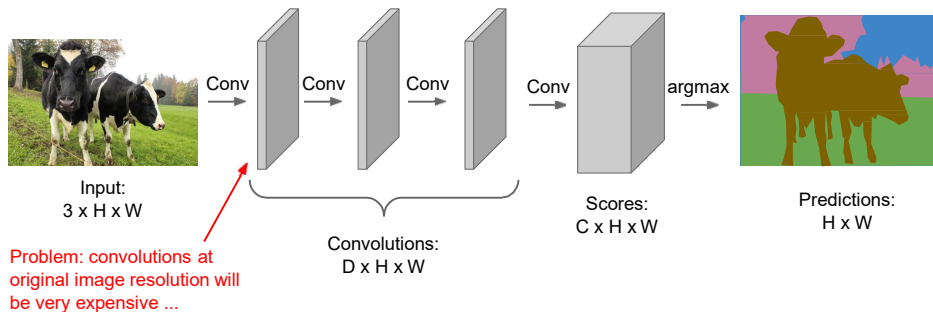


SCHOOL OF INFORMATION AND COMMUNICATION TECHNOLOGY

9

## Semantic Segmentation Idea: Fully Convolutional

Design a network as a bunch of convolutional layers to make predictions for pixels all at once!

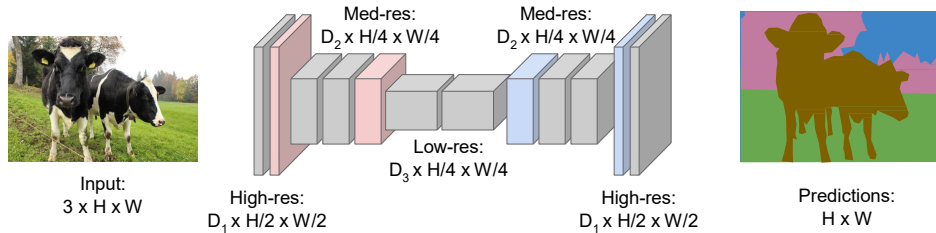


SCHOOL OF INFORMATION AND COMMUNICATION TECHNOLOGY

10

## Semantic Segmentation Idea: Fully Convolutional

Design network as a bunch of convolutional layers, with **downsampling** and **upsampling** inside the network!



Long, Shelhamer, and Darrell, "Fully Convolutional Networks for Semantic Segmentation", CVPR 2015  
 Noh et al, "Learning Deconvolution Network for Semantic Segmentation", ICCV 2015



SCHOOL OF INFORMATION AND COMMUNICATION TECHNOLOGY

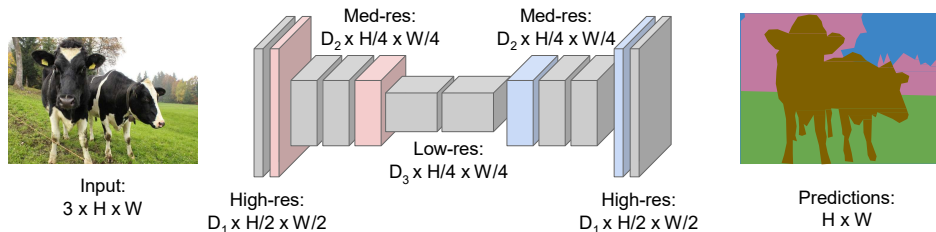
11

## Semantic Segmentation Idea: Fully Convolutional

**Downsampling:**  
 Pooling, strided  
 convolution

Design network as a bunch of convolutional layers, with **downsampling** and **upsampling** inside the network!

**Upsampling:**  
 ???



Long, Shelhamer, and Darrell, "Fully Convolutional Networks for Semantic Segmentation", CVPR 2015  
 Noh et al, "Learning Deconvolution Network for Semantic Segmentation", ICCV 2015



SCHOOL OF INFORMATION AND COMMUNICATION TECHNOLOGY

12

## In-Network upsampling: “Unpooling”

**Nearest Neighbor**

|   |   |
|---|---|
| 1 | 2 |
| 3 | 4 |

Input: 2 x 2

|   |   |   |   |
|---|---|---|---|
| 1 | 1 | 2 | 2 |
| 1 | 1 | 2 | 2 |
| 3 | 3 | 4 | 4 |
| 3 | 3 | 4 | 4 |

Output: 4 x 4

**“Bed of Nails”**

|   |   |
|---|---|
| 1 | 2 |
| 3 | 4 |

Input: 2 x 2

|   |   |   |   |
|---|---|---|---|
| 1 | 0 | 2 | 0 |
| 0 | 0 | 0 | 0 |
| 3 | 0 | 4 | 0 |
| 0 | 0 | 0 | 0 |

Output: 4 x 4



SCHOOL OF INFORMATION AND COMMUNICATION TECHNOLOGY

13

## In-Network upsampling: “Max Unpooling”

**Max Pooling**

Remember which element was max!

|   |   |   |   |
|---|---|---|---|
| 1 | 2 | 6 | 3 |
| 3 | 5 | 2 | 1 |
| 1 | 2 | 2 | 1 |
| 7 | 3 | 4 | 8 |

Input: 4 x 4

|   |   |
|---|---|
| 5 | 6 |
| 7 | 8 |

Output: 2 x 2

Rest of the network

**Max Unpooling**

Use positions from pooling layer

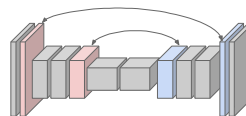
|   |   |
|---|---|
| 1 | 2 |
| 3 | 4 |

Input: 2 x 2

|   |   |   |   |
|---|---|---|---|
| 0 | 0 | 2 | 0 |
| 0 | 1 | 0 | 0 |
| 0 | 0 | 0 | 0 |
| 3 | 0 | 0 | 4 |

Output: 4 x 4

Corresponding pairs of  
downsampling and  
upsampling layers

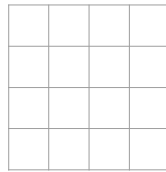


SCHOOL OF INFORMATION AND COMMUNICATION TECHNOLOGY

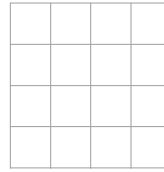
14

## Learnable Upsampling: Transpose Convolution

**Recall:** Normal 3 x 3 convolution, stride 1 pad 1



Input: 4 x 4



Output: 4 x 4

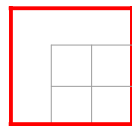


SCHOOL OF INFORMATION AND COMMUNICATION TECHNOLOGY

15

## Learnable Upsampling: Transpose Convolution

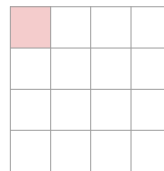
**Recall:** Normal 3 x 3 convolution, stride 1 pad 1



Input: 4 x 4



Dot product  
between filter  
and input



Output: 4 x 4

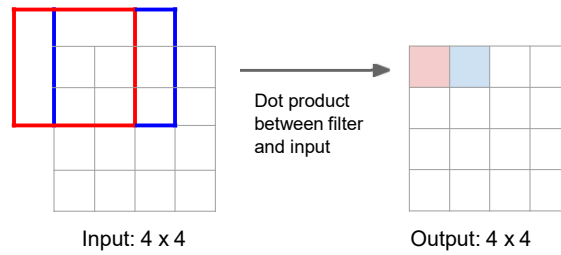


SCHOOL OF INFORMATION AND COMMUNICATION TECHNOLOGY

16

## Learnable Upsampling: Transpose Convolution

**Recall:** Normal 3 x 3 convolution, stride 1 pad 1

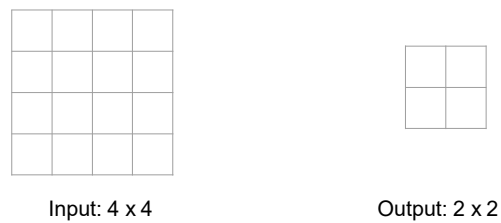


SCHOOL OF INFORMATION AND COMMUNICATION TECHNOLOGY

17

## Learnable Upsampling: Transpose Convolution

**Recall:** Normal 3 x 3 convolution, stride 2 pad 1

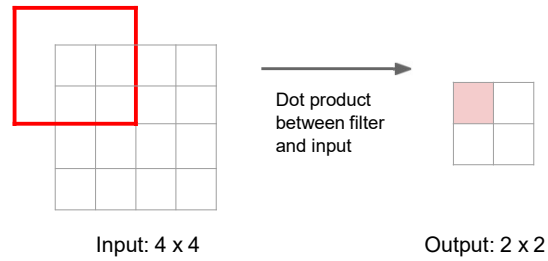


SCHOOL OF INFORMATION AND COMMUNICATION TECHNOLOGY

18

## Learnable Upsampling: Transpose Convolution

**Recall:** Normal 3 x 3 convolution, stride 2 pad 1

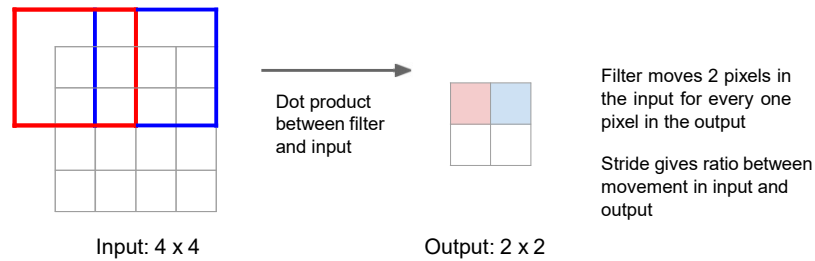


SCHOOL OF INFORMATION AND COMMUNICATION TECHNOLOGY

19

## Learnable Upsampling: Transpose Convolution

**Recall:** Normal 3 x 3 convolution, stride 2 pad 1



SCHOOL OF INFORMATION AND COMMUNICATION TECHNOLOGY

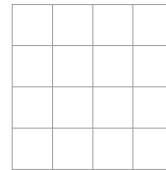
20

## Learnable Upsampling: Transpose Convolution

3 x 3 **transpose** convolution, stride 2 pad 1



Input: 2 x 2



Output: 4 x 4



SCHOOL OF INFORMATION AND COMMUNICATION TECHNOLOGY

21

## Learnable Upsampling: Transpose Convolution

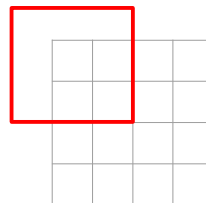
3 x 3 **transpose** convolution, stride 2 pad 1



Input: 2 x 2



Input gives  
weight for  
filter



Output: 4 x 4

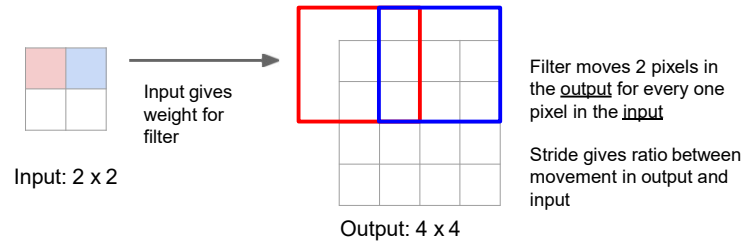


SCHOOL OF INFORMATION AND COMMUNICATION TECHNOLOGY

22

## Learnable Upsampling: Transpose Convolution

3 x 3 **transpose** convolution, stride 2 pad 1

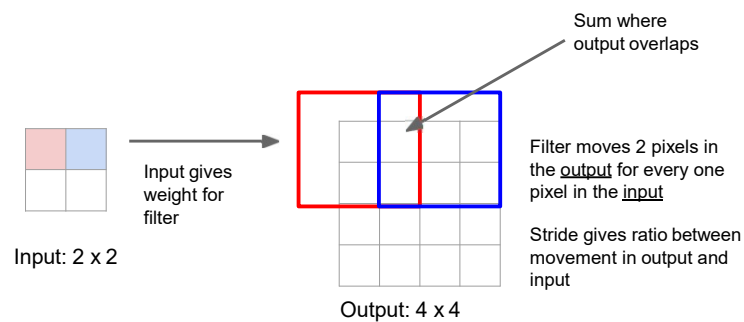


SCHOOL OF INFORMATION AND COMMUNICATION TECHNOLOGY

23

## Learnable Upsampling: Transpose Convolution

3 x 3 **transpose** convolution, stride 2 pad 1



SCHOOL OF INFORMATION AND COMMUNICATION TECHNOLOGY

24

## Learnable Upsampling: Transpose Convolution

3 x 3 **transpose** convolution, stride 2 pad 1

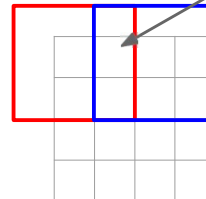
### Other names:

- Deconvolution (bad)
- Upconvolution
- Fractionally strided convolution
- Backward strided convolution



Input: 2 x 2

Input gives weight for filter



Output: 4 x 4

Sum where output overlaps

Filter moves 2 pixels in the output for every one pixel in the input

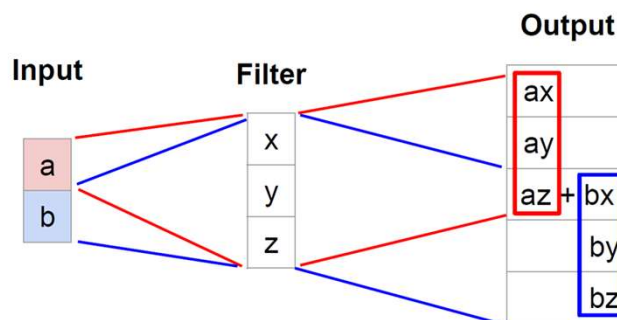
Stride gives ratio between movement in output and input



SCHOOL OF INFORMATION AND COMMUNICATION TECHNOLOGY

25

## Learnable Upsampling: Transpose Convolution



Output contains copies of the filter weighted by the input, summing at where at overlaps in the output

Need to crop one pixel from output to make output exactly 2x input



SCHOOL OF INFORMATION AND COMMUNICATION TECHNOLOGY

26

## Convolution as Matrix Multiplication (1D Example)

We can express convolution in terms of a matrix multiplication

$$\vec{x} * \vec{a} = X\vec{a}$$

$$\begin{bmatrix} x & y & z & 0 & 0 & 0 \\ 0 & x & y & z & 0 & 0 \\ 0 & 0 & x & y & z & 0 \\ 0 & 0 & 0 & x & y & z \end{bmatrix} \begin{bmatrix} 0 \\ a \\ b \\ c \\ d \\ 0 \end{bmatrix} = \begin{bmatrix} ay + bz \\ ax + by + cz \\ bx + cy + dz \\ cx + dy \end{bmatrix}$$

Example: 1D conv, kernel size=3, stride=1, padding=1



SCHOOL OF INFORMATION AND COMMUNICATION TECHNOLOGY

27

## Convolution as Matrix Multiplication (1D Example)

We can express convolution in terms of a matrix multiplication

$$\vec{x} * \vec{a} = X\vec{a}$$

$$\begin{bmatrix} x & y & z & 0 & 0 & 0 \\ 0 & x & y & z & 0 & 0 \\ 0 & 0 & x & y & z & 0 \\ 0 & 0 & 0 & x & y & z \end{bmatrix} \begin{bmatrix} 0 \\ a \\ b \\ c \\ d \\ 0 \end{bmatrix} = \begin{bmatrix} ay + bz \\ ax + by + cz \\ bx + cy + dz \\ cx + dy \end{bmatrix}$$

Example: 1D conv, kernel size=3, stride=1, padding=1

Convolution transpose multiplies by the transpose of the same matrix:

$$\vec{x} *^T \vec{a} = X^T \vec{a}$$

$$\begin{bmatrix} x & 0 & 0 & 0 \\ y & x & 0 & 0 \\ z & y & x & 0 \\ 0 & z & y & x \\ 0 & 0 & z & y \\ 0 & 0 & 0 & z \end{bmatrix} \begin{bmatrix} a \\ b \\ c \\ d \end{bmatrix} = \begin{bmatrix} ax \\ ay + bx \\ az + by + cx \\ bz + cy + dx \\ cz + dy \\ dz \end{bmatrix}$$

When stride=1, convolution transpose is just a regular convolution (with different padding rules)



SCHOOL OF INFORMATION AND COMMUNICATION TECHNOLOGY

28

## Convolution as Matrix Multiplication (1D Example)

We can express convolution in terms of a matrix multiplication

$$\vec{x} * \vec{a} = X\vec{a}$$

$$\begin{bmatrix} x & y & z & 0 & 0 & 0 \\ 0 & 0 & x & y & z & 0 \end{bmatrix} \begin{bmatrix} 0 \\ a \\ b \\ c \\ d \\ 0 \end{bmatrix} = \begin{bmatrix} ay + bz \\ bx + cy + dz \end{bmatrix}$$

Example: 1D conv, kernel size=3, stride=2, padding=1



SCHOOL OF INFORMATION AND COMMUNICATION TECHNOLOGY

29

## Convolution as Matrix Multiplication (1D Example)

We can express convolution in terms of a matrix multiplication

$$\vec{x} * \vec{a} = X\vec{a}$$

$$\begin{bmatrix} x & y & z & 0 & 0 & 0 \\ 0 & 0 & x & y & z & 0 \end{bmatrix} \begin{bmatrix} 0 \\ a \\ b \\ c \\ d \\ 0 \end{bmatrix} = \begin{bmatrix} ay + bz \\ bx + cy + dz \end{bmatrix}$$

Example: 1D conv, kernel size=3, stride=2, padding=1

Convolution transpose multiplies by the transpose of the same matrix:

$$\vec{x} *^T \vec{a} = X^T \vec{a}$$

$$\begin{bmatrix} x & 0 \\ y & 0 \\ z & x \\ 0 & y \\ 0 & z \\ 0 & 0 \end{bmatrix} \begin{bmatrix} a \\ b \end{bmatrix} = \begin{bmatrix} ax \\ ay \\ az + bx \\ by \\ bz \\ 0 \end{bmatrix}$$

When stride>1, convolution transpose is no longer a normal convolution!



SCHOOL OF INFORMATION AND COMMUNICATION TECHNOLOGY

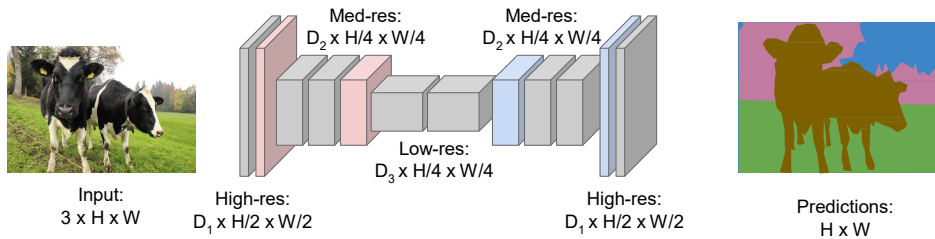
30

## Semantic Segmentation Idea: Fully Convolutional

**Downsampling:**  
Pooling, strided  
convolution

Design network as a bunch of convolutional layers, with **downsampling** and **upsampling** inside the network!

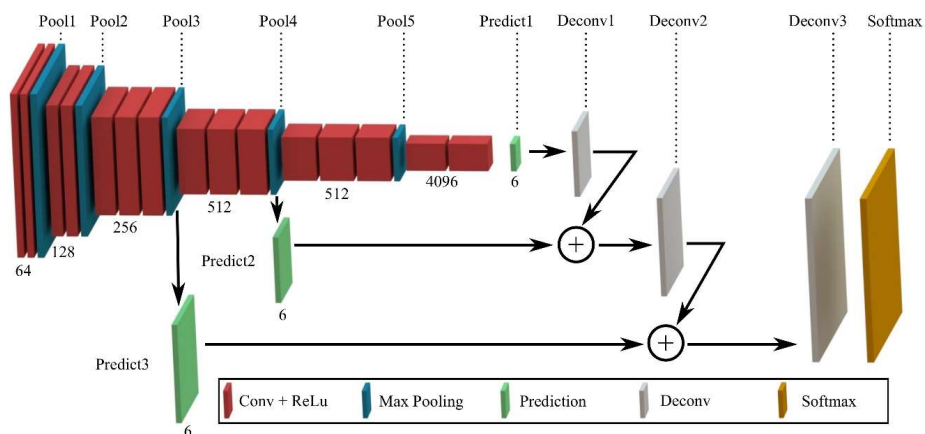
**Upsampling:**  
Unpooling or strided  
transpose convolution



SCHOOL OF INFORMATION AND COMMUNICATION TECHNOLOGY

31

## Semantic Segmentation: FCN with skip connections

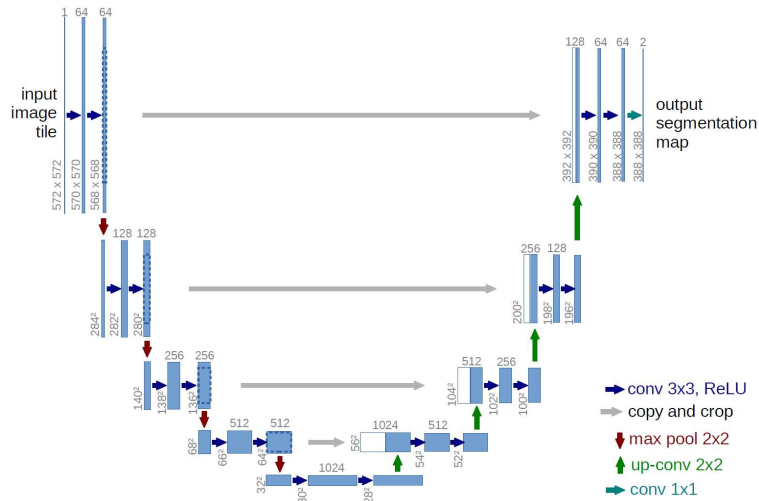


Long et al. Fully Convolutional Networks for Semantic Segmentation, CVPR 2015

SCHOOL OF INFORMATION AND COMMUNICATION TECHNOLOGY

32

## Unet and ...

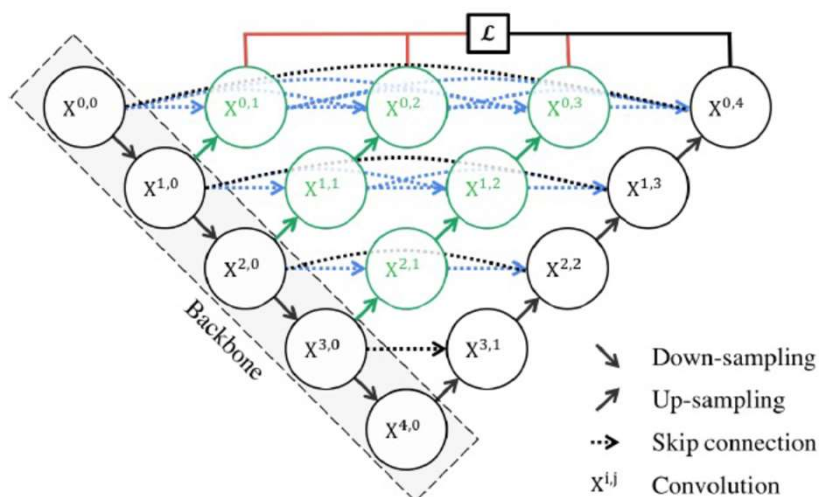


O. Ronneberger et al. U-Net: Convolutional Networks for Biomedical Image Segmentation, MICCAI 2015

SCHOOL OF INFORMATION AND COMMUNICATION TECHNOLOGY

33

## Unet++



Zhou et al. UNet++: Redesigning Skip Connections to Exploit Multiscale Features in Image Segmentation. Journal of IEEE Transactions on Medical Imaging, 2020

SCHOOL OF INFORMATION AND COMMUNICATION TECHNOLOGY

34

## Metrics for Segmentation Models

- **Pixel Accuracy (PA):**

$$PA = \frac{\sum_{i=0}^K p_{ii}}{\sum_{i=0}^K \sum_{j=0}^K p_{ij}}$$

- The ratio of pixels properly classified, divided by the total number of pixels.

- **Mean Pixel Accuracy (MPA):**

$$MPA = \frac{1}{K+1} \sum_{i=0}^K \frac{p_{ii}}{\sum_{j=0}^K p_{ij}}$$

- The ratio of correct pixels is computed in a per-class manner and then averaged over the total number of classes.



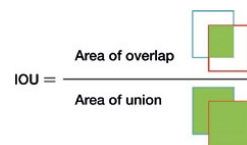
SCHOOL OF INFORMATION AND COMMUNICATION TECHNOLOGY

35

## Metrics for Segmentation Models

- **Intersection over Union (IoU):**

$$IoU = J(A, B) = \frac{|A \cap B|}{|A \cup B|}$$



- Also called **Jaccard Index**
- The most commonly used metrics in semantic segmentation. (mean-IoU/mIoU)
- A denotes the ground truth and B denotes the predicted segmentation maps.



SCHOOL OF INFORMATION AND COMMUNICATION TECHNOLOGY

36

## Metrics for Segmentation Models

- Precision / Recall / F1 score:

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}$$

$$\text{F1-score} = \frac{2 \text{ Prec Rec}}{\text{Prec} + \text{Rec}}$$

F1: harmonic mean of precision and recall

|                                            |           | Actual Value<br>(as confirmed by experiment) |                                |
|--------------------------------------------|-----------|----------------------------------------------|--------------------------------|
|                                            |           | positives                                    | negatives                      |
| Predicted Value<br>(predicted by the test) | positives | <b>TP</b><br>True<br>Positive                | <b>FP</b><br>False<br>Positive |
|                                            | negatives | <b>FN</b><br>False<br>Negative               | <b>TN</b><br>True<br>Negative  |



SCHOOL OF INFORMATION AND COMMUNICATION TECHNOLOGY

37

## Metrics for Segmentation Models

- Dice coefficient:

$$\text{Dice} = \frac{2|A \cap B|}{|A| + |B|}$$

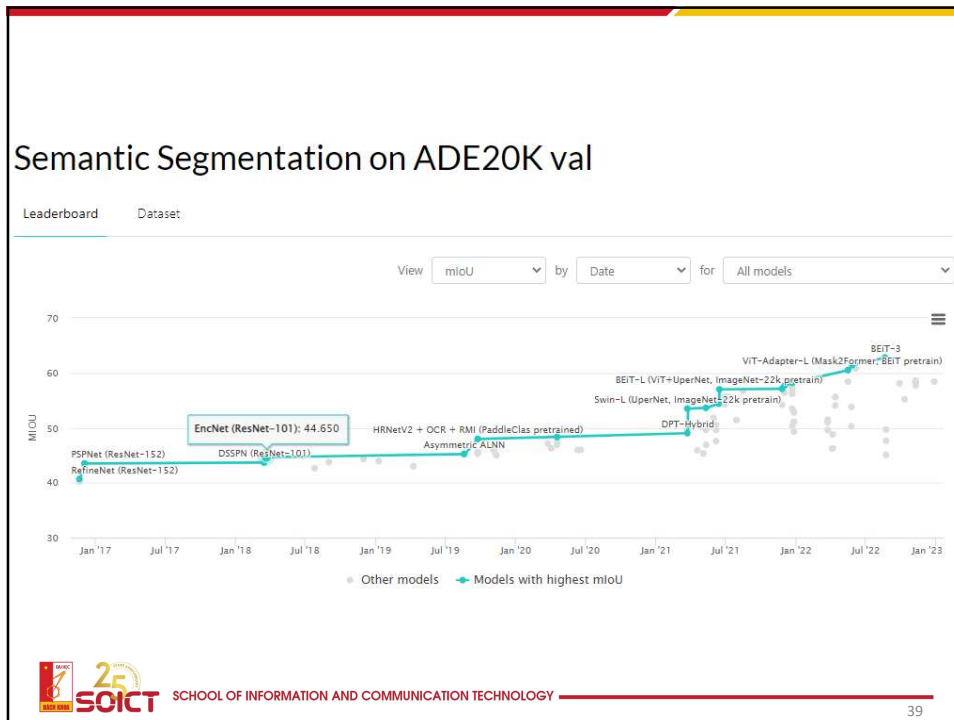
$$\text{Dice} = \frac{2TP}{2TP + FP + FN} = F1$$

- Essentially identical to the F1 score.
- The Dice coefficient and IoU are positively correlated.



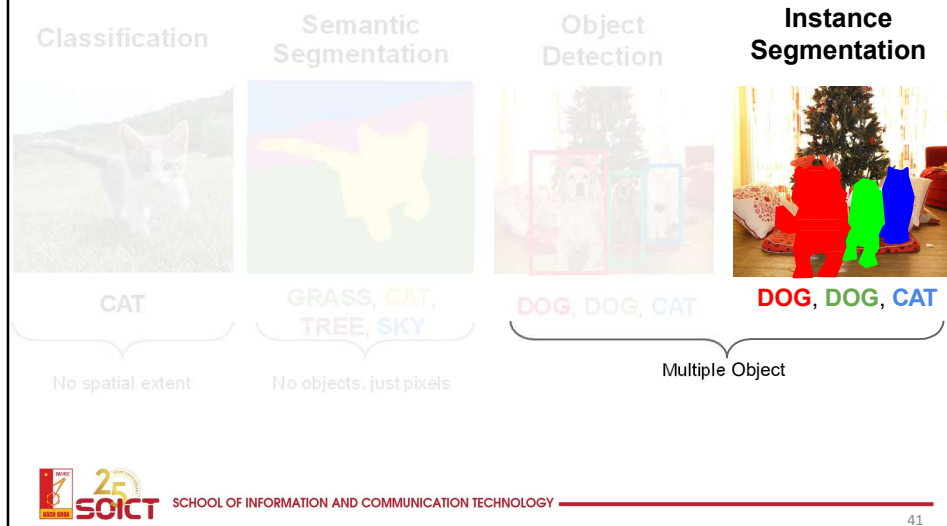
SCHOOL OF INFORMATION AND COMMUNICATION TECHNOLOGY

38

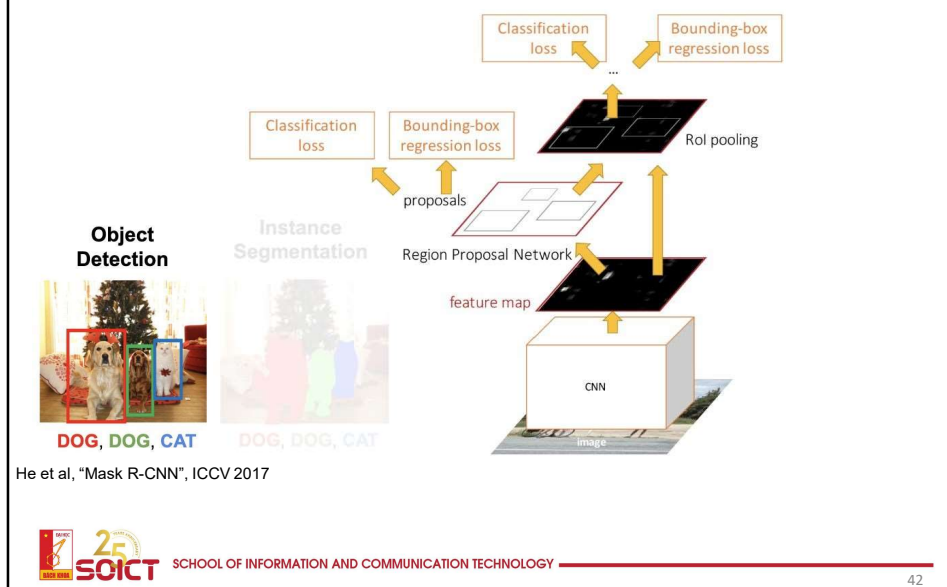


## Instance segmentation

## Instance Segmentation

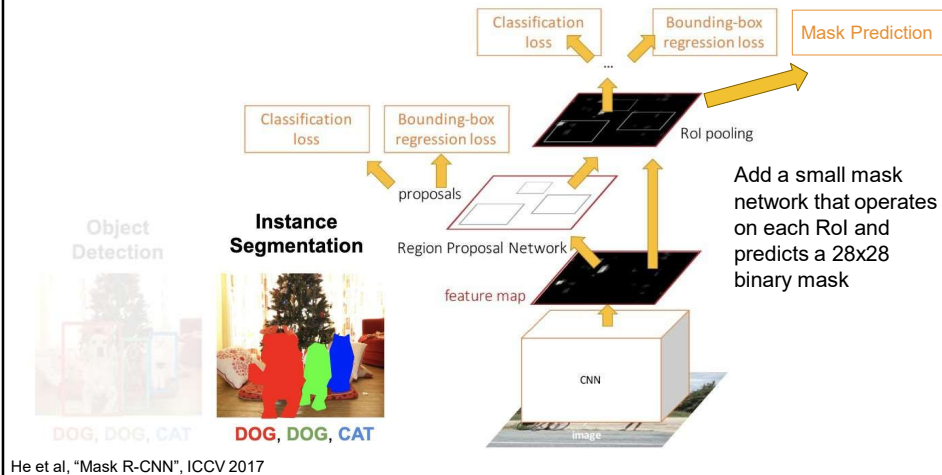


## Object Detection: Faster R-CNN



He et al, "Mask R-CNN", ICCV 2017

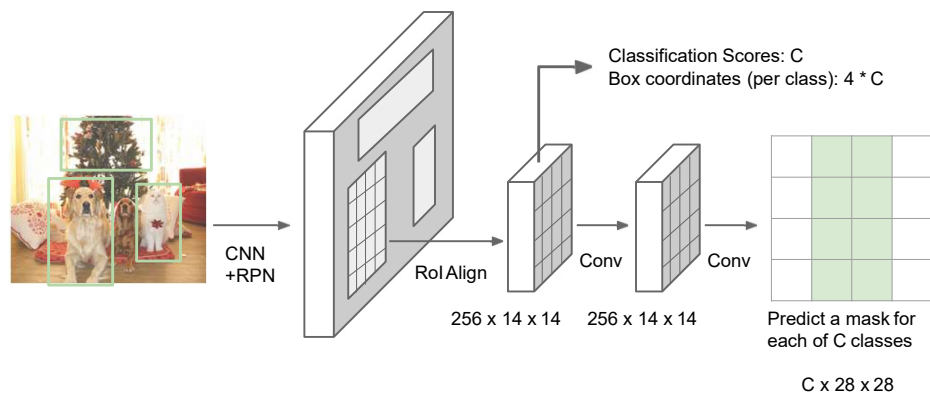
## Instance Segmentation: Mask R-CNN



SCHOOL OF INFORMATION AND COMMUNICATION TECHNOLOGY

43

## Mask R-CNN



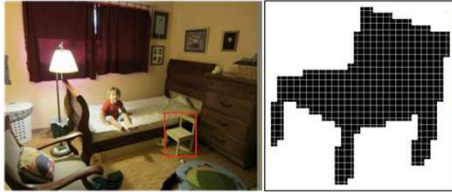
He et al, "Mask R-CNN", ICCV 2017



SCHOOL OF INFORMATION AND COMMUNICATION TECHNOLOGY

44

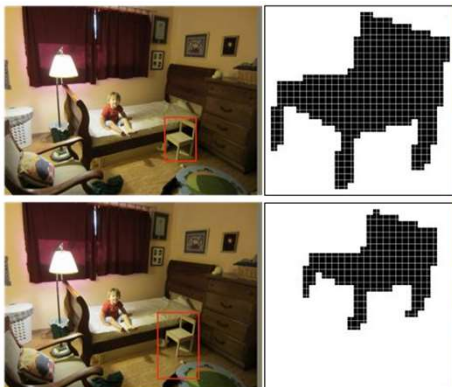
## Mask R-CNN: Example Mask Training Targets



SCHOOL OF INFORMATION AND COMMUNICATION TECHNOLOGY

45

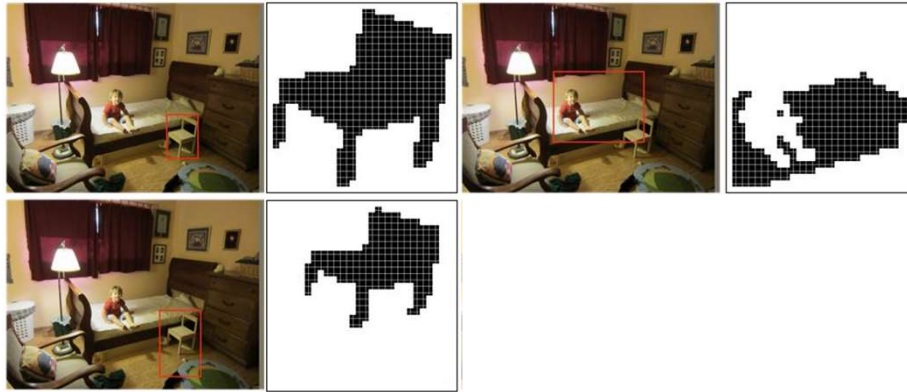
## Mask R-CNN: Example Mask Training Targets



SCHOOL OF INFORMATION AND COMMUNICATION TECHNOLOGY

46

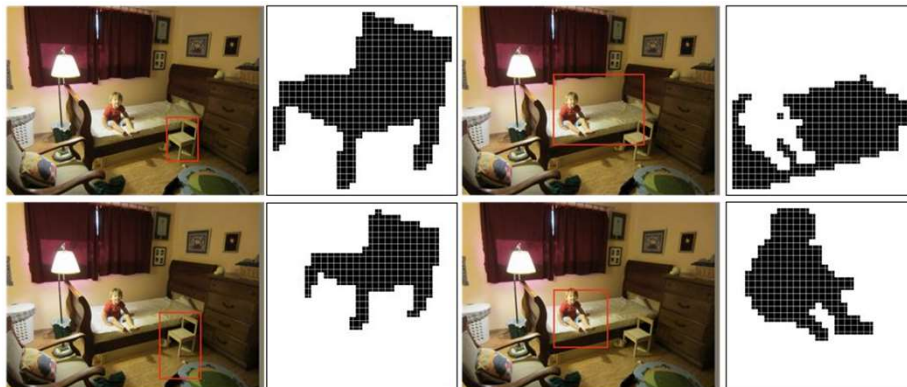
## Mask R-CNN: Example Mask Training Targets



SCHOOL OF INFORMATION AND COMMUNICATION TECHNOLOGY

47

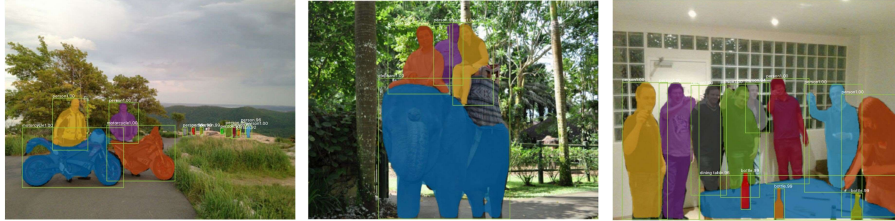
## Mask R-CNN: Example Mask Training Targets



SCHOOL OF INFORMATION AND COMMUNICATION TECHNOLOGY

48

## Mask R-CNN: Very Good Results!



He et al, "Mask R-CNN", ICCV 2017



SCHOOL OF INFORMATION AND COMMUNICATION TECHNOLOGY

49

## Mask R-CNN Also does pose

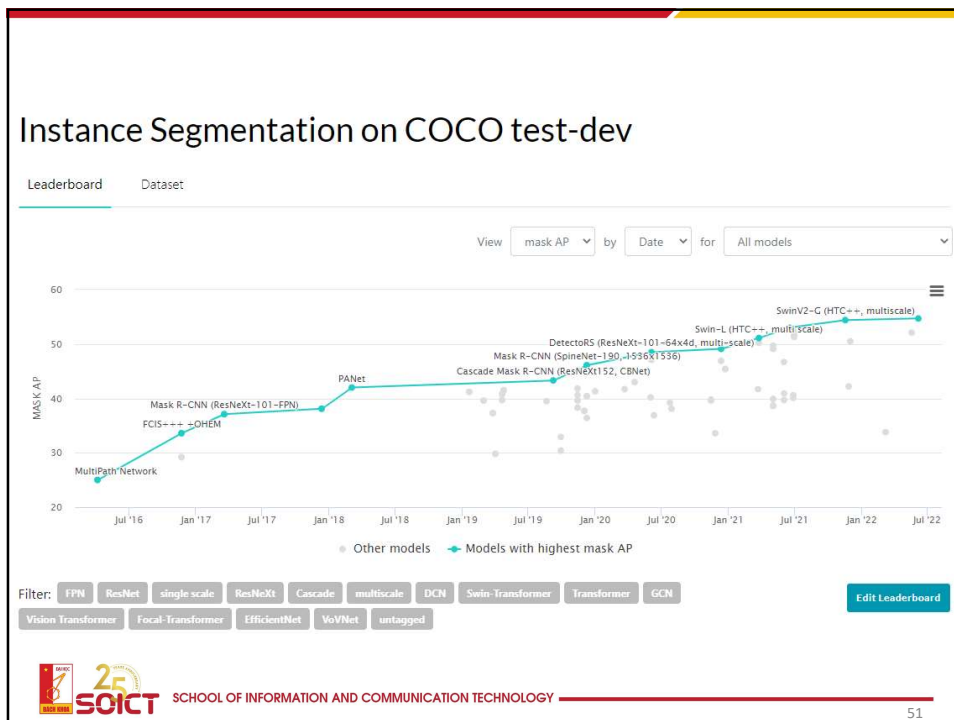


He et al, "Mask R-CNN", ICCV 2017



SCHOOL OF INFORMATION AND COMMUNICATION TECHNOLOGY

50



## And ...

- Transfer learning
- Knowledge distillation
- Domain adaptation / domain generalization
- Semi-supervised

## Open Source Frameworks

- Lots of good implementations on GitHub!
- TensorFlow Detection API:
  - [https://github.com/tensorflow/models/tree/master/research/object\\_detection](https://github.com/tensorflow/models/tree/master/research/object_detection)
 Faster RCNN, SSD, RFCN, Mask R-CNN
- Caffe2 Detectron:
  - <https://github.com/facebookresearch/Detectron>
 Mask R-CNN, RetinaNet, Faster R-CNN, RPN, Fast R-CNN, R-FCN
- Finetune on your own dataset with pre-trained models



## References

1. CS231n: Convolutional Neural Networks for Visual Recognition  
<http://cs231n.stanford.edu/>
2. Object Detection Creation from Scratch. Samsung R&D Institute Ukraine. Vitaliy Bulygin  
<https://aiukraine.com/wp-content/uploads/2018/08/Vitalij-Buly-gin-.pptx>
3. CornerNet: Detecting Objects as Paired Keypoints  
<https://pvl.cs.princeton.edu/assets/CornerNet.pptx>



